Project - Low Level Design

on

Real Estate Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Samruddhi Muley	EN22CS304056
2	Payal Singh	EN22CS301701
3	Aarti Bamanke	EN22EE301001
4	Akshat Joshi	EN22CS304006
5	Punit Pawar	EN22CS303038
6	Shubhi Dubey	EN22CS304061

Group Name: 01D11

Project Number: AAI-27

Industry Mentor Name:

University Mentor Name: Dr. Nishant Shrivastava

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

The Real Estate Triage Agent is an AI-powered system designed to automate the initial handling of real estate support communications. It uses Large Language Models (LLMs) to analyze incoming customer messages, classify their urgency and intent, extract key information, and generate professional draft responses.

The system improves response time, reduces manual workload, and ensures structured handling of support queries. It follows an agent-based architecture using LangChain's ReAct framework to dynamically decide actions.

1.1 Scope of the document

This document provides the Low-Level Design (LLD) of the Real Estate Triage Agent. It includes:

- Detailed component-level architecture
- Internal working of modules and tools
- Data flow and processing logic
- API specifications
- Design decisions and constraints

This document is intended for developers, testers, and system maintainers.

1.2 Intended Audience

This document is intended for:

- Developers implementing backend logic
- QA engineers testing the system
- Trainers and evaluators reviewing the project
- Future contributors extending the system

1.3 System Overview

The system processes real estate support messages and performs:

1. Classification → Determines urgency and intent
2. NER Extraction → Extracts entities like property ID, name, date
3. Response Generation → Creates a professional reply
4. Report Generation → Builds structured JSON output

The application is built using:

- Python
- LangChain (ReAct Agent)
- Groq LLM API
- Flask (for API layer)

Technology	Function	Simple analogy
Python Base language	Glue that holds all components together	The kitchen where cooking happens
LangChain / CrewAI Agent framework	Runs the agent loop, picks and calls tools	The manager who assigns tasks to workers
Groq API + Llama LLM provider	The brain — reads text, reasons, writes replies	The smart employee who actually does the thinking
spaCy / LLM NER Entity extraction	Finds names, IDs, dates inside text	A highlighter that marks key facts in a document
FastAPI / CLI Entry point	Receives incoming messages, returns output	The front door of the office
Python dict / JSON Output format	Packages the triage result for the human	The completed report handed to the team

2. System Design

2.1 Application Design

The system follows a layered architecture:

Layer 1: Input Layer

- Accepts raw messages (string input)
- Sources: UI, API, or test scripts

Layer 2: Agent Core

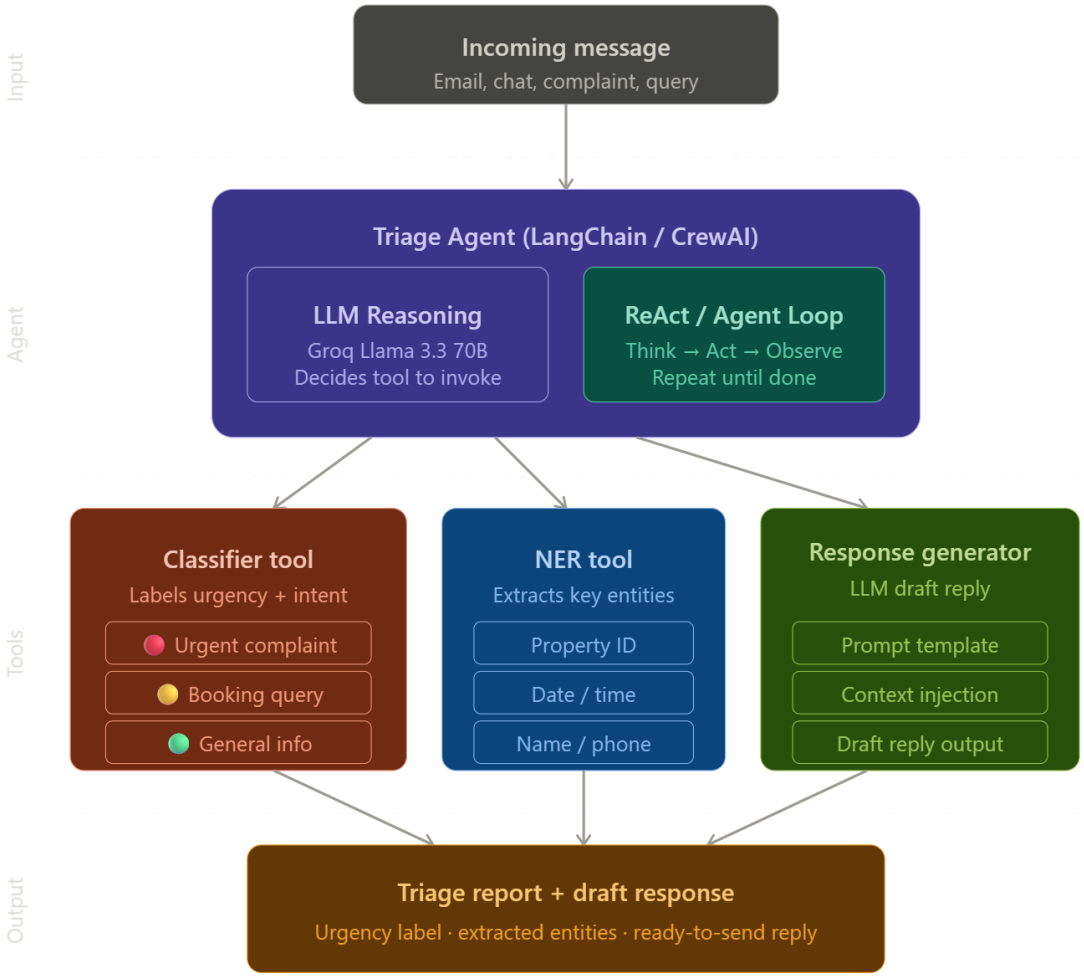
- Uses LangChain ReAct Agent
- Implements Think → Act → Observe loop
- Decides which tool to call

Layer 3: Tools Layer

- Classifier Tool
- NER Extraction Tool
- Response Generator Tool

Layer 4: Output Layer

- Generates structured JSON report
- Includes routing and SLA details



2.2 Process Flow

1. User inputs a message
2. Message is sent to Agent Core
3. Agent executes:
   - Step 1 → Classification
   - Step 2 → NER Extraction
   - Step 3 → Response Generation
4. Results are passed to report builder
5. Routing & SLA are determined
6. Final JSON report is generated
7. Response is returned to UI/APIworking

2.3 Information Flow

User Input → Agent Core
→ Classifier Tool → Output (urgency, intent)
→ NER Tool → Output (entities)
→ Response Tool → Draft Response
→ Report Builder → Final JSON
→ API/UI Response

2.4 Components Design

1. Config Module

- Initializes Groq LLM
- Reads API key from environment variables

2. Classifier Tool

- **Function:** classify_message(message)
- **Input:** Raw message
- **Output**:

```
{
  "urgency": "urgent",
  "intent": "complaint"
}
```

- **Logic:**
  - Uses prompt engineering
  - Enforces strict JSON format

3. NER Extractor

- **Function**: extract_entities(message)
- **Output**:{

```
  "property_ids": [],
  "names": [],
  "dates": [],
  "phone_numbers": [],
  "locations": [],
  "amounts": []
}
```

4. Response Generator

Function: generate_response(message, classification, entities)

- Builds context-aware prompt
- Uses tone rules based on urgency
- Generates 3–5 line response

5. Agent Core

File: triage_agent.py

- Uses LangChain ReAct agent
- Dynamically selects tools
- Maintains reasoning loop

6. Report Builder

- **Function:** build_triage_report(message)
- **Output Structure:**

```
{
  "triage_report": {
    "metadata": {},
    "input": {},
    "classification": {},
    "entities": {},
    "routing": {},
    "draft_response": ""
  }
}
```

2.5 Key Design Considerations

1. Modularity

- Each tool is independent and reusable

2. Scalability

- Can be extended with more tools

3. Error Handling

- JSON parsing safeguards
- Default fallbacks

4. Performance

- Groq used for low latency

5. Maintainability

- Clean separation of concerns

2.6 API Catalogue

1. Health Check

Endpoint: /

Method: GET

Response:

```
{
  "status": "Running"
}
```

2. Triage API

Endpoint: /triage

Method: POST

Request:

```
{
  "message": "My flat has leakage..."
}
```

Response:

```
{
  "triage_report": { ... }
}
```

---

3. References

- LangChain Documentation
- Groq API Docs
- Python Official Docs
- spaCy NER Documentation